

ChaosArena v1-album-store - Report

1. Roughly how many submissions did it take before you passed all critical scenarios, and what was the most common failure?

It took about 4-5 submissions before all critical scenarios passed. The most common failure was PostgreSQL permission errors. After creating the database and tables, the `albumuser` didn't have `GRANT ALL` on the tables, causing `permission denied for table albums` (SQLSTATE 42501) which made every write operation return 500.

2. Where are your photo files stored, and why did you pick that over other options?

Photos are stored in AWS S3 with public-read ACL. I chose S3 because it provides durable, scalable object storage accessible via public URLs without needing to serve files from the application server. Local disk storage would not survive instance restarts and would be inaccessible across multiple instances in my ALB setup.

3. Describe your deployment setup - how many instances, what cloud services, and how they connect to each other.

Two t3.large EC2 instances sit behind an Application Load Balancer. EC2-1 runs both the Go API and PostgreSQL 15; EC2-2 runs only the Go API and connects to EC2-1's PostgreSQL over the VPC private network (172.31.x.x:5432). Both instances upload photos to the same S3 bucket via an S3 VPC Gateway Endpoint for faster private-network transfers. Terraform manages all infrastructure.

4. Did you use a reverse proxy or load balancer? If so, what role does it play in your architecture?

Yes, an AWS ALB on port 80 round-robins requests across both EC2 instances on port 8080. Its primary role is distributing concurrent file uploads across instances so each handles roughly half the traffic, effectively doubling the aggregate network bandwidth and S3 upload capacity. This was the single biggest architectural win. S15 (large payload) `accept_p95` dropped from 11.3s to 5.5s.

5. How does your background worker get notified that there's a new photo to process? Did you use a queue, polling, or something else?

An in-memory buffered Go channel acts as the queue. The HTTP handler submits a `Job` struct (containing the photo bytes and an S3 upload closure) into the channel, and 100 worker goroutines drain it. A memory semaphore (capacity 25) limits concurrent S3 uploads to prevent OOM. This is faster than SQS (roughly 0ms vs 5-20ms publish latency) which matters for ChaosArena scoring.

6. The spec requires that `seq` is assigned in the POST handler, not the background worker. Why does that matter, and how did you ensure correctness under concurrent uploads to the same album?

Assigning `seq` in the handler guarantees the client receives a deterministic, gap-free sequence number in the 202 response immediately, before any async processing. I use a single PostgreSQL transaction that atomically does `UPDATE albums SET photo_seq = photo_seq + 1 RETURNING photo_seq` followed by the photo INSERT. PostgreSQL's row-level lock on the album row serializes concurrent uploads to the same album, ensuring unique, sequential numbers even under heavy concurrency.

7. What happens in your system if the worker crashes or fails halfway through processing a photo?

The photo row remains in `status = 'processing'` permanently, becoming a stale record. If the S3 upload fails but the worker itself is alive, the worker catches the error and updates the status to `'failed'`. On graceful shutdown (SIGTERM), the server drains the worker pool before exiting to avoid losing queued jobs.

8. What does your database schema look like? What tables or collections did you create and why?

Two tables: `albums` (`album_id` TEXT PK, `title`, `description`, `owner`, `photo_seq` BIGINT DEFAULT 0) and `photos` (`photo_id` TEXT PK, `album_id` FK referencing albums, `seq` BIGINT, `status` TEXT, `s3_key` TEXT, `url` TEXT) with a UNIQUE constraint on (`album_id`, `seq`). The `photo_seq` counter on albums enables atomic sequence assignment without a separate sequence table.

10. Which load testing scenario was the hardest for you, and what bottleneck did you discover?

S12 (concurrent photo uploads) was the hardest, still only scoring 5/15 at p95 of about 6000ms. The bottleneck is S3 upload latency: each individual upload takes 5-6 seconds regardless of concurrency or instance count. Unlike S15 where I halved accept time with 2 instances, S12 measures per-upload completion time which cannot be improved by horizontal scaling alone.

11. What was the single most impactful change you made to improve your load test scores?

Adding a second EC2 instance behind an ALB. This single change jumped the score from 169 to 176 (+7 points). S15 `accept_p95` was cut in half (11.3s to 5.5s) because large file uploads were distributed across two instances, each with its own 5 Gbps network bandwidth.

15. If you had another week, what is the one thing you would change or add to your system to improve your score?

I would implement S3 Transfer Acceleration or experiment with pre-signed URL uploads where the client uploads directly to S3, bypassing the server entirely. The remaining 15 points are almost entirely bounded by S3 upload latency (about 6s per file). Eliminating the server-as-proxy pattern could bring S12 p95 under 1 second.